Planora AI — Build Charter & Reality Matrix

Contents

Version: 1.0 · **Date:** 2026-07-05 · **Owner:** Abhijit (Founder) **Purpose:** The single source of truth for what Planora *actually is today*, what it's missing to become a production-grade enterprise product, and the sequenced strategy to build it. This document supersedes the README and all prior status docs, which are inaccurate (see §1).

> **How to use this doc:** This is your development charter. When you or an engineer is about to build something, it must map to a phase in §6 and move a module from a lower reality grade (§4) toward "REAL," or close a foundation gap (§5). If a proposed piece of work does neither, it's out of scope for now. Re-grade the matrix (§4, §10) at the end of every phase.

---

1. Why the existing docs are wrong (read this first)

The README's "Capability Matrix" marks all 16 module areas as **"Strong."** Verified against the code, that is false and dangerous — it will blow up in a technical diligence call or an enterprise security review. Ground-truth findings:

- **Authentication does not exist on the backend.** All 82 API endpoints are open — no auth dependency on any route. `passlib` and `python-jose` are listed in `requirements.txt` but never imported or used. Login is fully mocked in the frontend (`AuthContext.tsx` -> `MOCK_USERS` with plaintext passwords like `admin123`).
- **There is no multi-tenancy.** No `organization_id` / `tenant_id` on any table in `models.py`. `orgId: 'org1'` is a hardcoded constant in the frontend only.
- **RBAC is client-side only** — enforced in React (`ROLE_RANK`, `ACTION_MIN_ROLE`), which means it's cosmetic. Anyone can hit the API directly.
- **Many "Strong" features are simulated** — connectors (SAP, Oracle, Manhattan, Coupa, ORTEC) are mock objects; dozens of metrics are `hash(sku) % 100` or `np.random` (52 mock/random markers in `main.py`).
- **Zero automated tests. No CI/CD.** SQLite by default, no migration tooling (Alembic). Redis is a dependency but unused.

What IS real and genuinely good: the forecasting engine (`forecasting.py`, `automl.py`, `ensembles.py` — 11 statistical models + sklearn/XGBoost/LightGBM/AutoML, real backtesting for MAE/RMSE/MAPE), the

inventory math (safety stock, ROP/EOQ, ABC/XYZ), anomaly detection (IsolationForest), retail clustering (K-means), and space optimization (SciPy LP). The domain modeling is excellent. **The problem is not the intelligence — it's the platform around it.**

---

2. Reality grading system

Every module and system in this doc is graded on how close it is to production:

Grade	Meaning
[REAL] **REAL**	Genuine computation on real/uploaded data; production-viable logic. Needs hardening, not invention.
[PARTIAL] **PARTIAL**	Real algorithm exists but runs on seeded/mock data, is incomplete, or blends real + mock outputs.
[MOCK] **SIMULATED**	UI shell over mock/`hash`/`random` outputs. No real computation behind it.
[BLOCKED] **FOUNDATION-BLOCKED**	Cannot be made real until multi-tenancy + real auth exist (§5.1, §5.2).

The honest headline: **one [REAL] engine (forecasting), a cluster of [PARTIAL] quantitative modules that are close, and a large [MOCK] demo surface — all sitting on an [BLOCKED] foundation.**

---

3. Current architecture (as-built)

```
Next.js 15 (App Router, TS, Recharts)          <- 12 feature modules, ~9.3k LOC
   |  fetch -> http://localhost:8000              <- client-side auth + RBAC (mock)
   |  AI Copilot -> user's own LLM API key (browser)  <- BYO-key passthrough, no server
   v
FastAPI (single process, sync)                  <- 82 endpoints, 3.8k-line main.py
   |  real ML: statsmodels / sklearn / xgboost / lightgbm / scipy
   |  NO auth · NO tenancy · NO background jobs · NO cache
   v
SQLite (default)  ·  SQLAlchemy ORM  ·  no migrations  ·  8 tables, no tenant column
```

Key structural constraints this imposes: - Forecasting runs **synchronously inside the HTTP request** — will time out on real multi-SKU datasets. - **No isolation** between users/orgs — a genuine data-leak risk the moment there are two customers. - **No horizontal scalability** — single process, no queue, no cache, in-file DB.

---

4. Module Reality Matrix

#	Module	Grade	What's real	What's mock / missing
1	**Demand Planning** (forecast, consensus, overrides, FVA)	[PARTIAL]->[REAL]	Forecasting engine, backtesting, override grid	Imports `mockData`; FVA uses `mock_mape` (hash); consensus not persisted per-tenant; no batch/scheduled runs
2	**Inventory Optimization** (safety stock, ROP/EOQ, ABC/XYZ, multi-echelon, working capital)	[PARTIAL]	Core formulas are real math	Runs on seeded data; multi-echelon partly synthetic; not wired to live inventory
3	**Supply Chain Diagnostics** (FVA, entropy, anomaly)	[PARTIAL]	IsolationForest anomaly detection, CV/entropy real	FVA MAPE mocked; needs stored forecast-vs-actual history
4	**S&OP / IBP**	[PARTIAL]	Reconciliation & RCCP math plausible	Cycle state is mock/stateless; not a real multi-user workflow
5	**Financial Planning** (P&L, cash flow, working capital)	[PARTIAL]	DIO/DSO/DPO & cash-conversion formulas real	Margins/growth are `hash()` mocks; not tied to real GL/actuals
6	**Scenario & Digital Twin** (Monte Carlo, demand shock)	[PARTIAL]	Monte Carlo & bullwhip sims are legitimate	Network/twin layout is static; not fed by real network data
7	**Pricing & Promotion** (elasticity, promo ROI, markdown)	[PARTIAL]	Elasticity/ROI math exists	Elasticity coefficients assumed/mock; no real price-response learning
8	**Retail & Assortment** (clustering, space opt)	[PARTIAL]	K-means + SciPy LP are real algorithms	Runs on seeded stores; not real POS/planogram data
9	**Supplier Collaboration / Portal**	[MOCK][BLOCKED]	UI + commit workflow shell	No real external-user auth; needs tenancy; largely mock

#	Module	Grade	What's real	What's mock / missing
10	**Executive Analytics / Control Tower**	[MOCK]	Chart/KPI UI	Imports `mockData`; supplier scorecards & many KPIs synthetic
11	**Self-Service BI** (query builder, semantic layer, lineage)	[MOCK]	Large, polished UI (2.6k LOC)	Semantic layer & data lineage are UI mockups, not engines
12	**Execution / Connectors** (ERP/WMS/TMS/procurement)	[MOCK]	Event stream reads real DB logs	All connectors simulated; "Force Sync" logs a fake event
13	**Governance / RBAC / Audit / MDM**	[PARTIAL][BLOCKED]	Audit-log model & MDM SKU registry real	RBAC frontend-only; governance settings partial; blocked on real auth

Reading the matrix: #1–#3 are your Primary loop and closest to [REAL] — that's where hardening effort concentrates. #4–#8 are [PARTIAL] quantitative modules to make real *after* the Primary loop sells. #9–#12 are [MOCK] demo surface — freeze them (keep for demos, stop investing) until the foundation and Primary loop are done.

---

5. Cross-cutting technical gaps (the real blockers)

These are not features — they're the floor beneath every feature. Nothing above [PARTIAL] is possible without them. Ordered by severity.

5.1 Multi-tenancy — [BLOCKED] CRITICAL

No org isolation anywhere. **Required:** add `organization_id` (FK) to every business table; enforce row-level filtering in a shared query layer (or a tenant-scoped DB session); tenant-aware object storage for uploads. *Nothing else in §6 Phase 0 matters more.*

5.2 Authentication & Authorization — [BLOCKED] CRITICAL (security hole)

Backend is fully open; auth is mocked in the browser. **Required:** server-side auth (JWT issuance + refresh, `passlib` bcrypt hashing — deps are already present), a `get_current_user` dependency on **every** route, server-enforced RBAC (move `ACTION_MIN_ROLE` logic to the API), and SSO/SAML on the roadmap for enterprise. Frontend RBAC stays as UX only.

5.3 Data layer & migrations — [MOCK]

SQLite + no migrations won't survive production. **Required:** PostgreSQL as the real target, Alembic migrations from day one, connection pooling, and re-evaluation of JSON columns (`hierarchy_levels`, `exogenous_variables`) that are hard to query/index at scale.

5.4 Async & background jobs — [MOCK]

Forecasting runs in-request and will time out. **Required:** a job queue (Celery or RQ on the Redis dep you already have), async batch forecasting across all SKUs, scheduled/recurring runs, and job status surfaced in the UI.

5.5 Testing & CI/CD — [MOCK]

Zero tests. **Required:** pytest for the backend (forecasting/inventory math first — these are your crown jewels and must be regression-locked), Vitest/Jest + Playwright for the frontend, and a CI pipeline (GitHub Actions) that blocks merges on failing tests. Set a coverage floor on the Primary-loop code paths.

5.6 Observability — [MOCK]

`logging_config.py` exists but there's no error tracking, metrics, or tracing. **Required:** structured logging, error tracking (Sentry), request/latency metrics, and health/readiness endpoints for deployment.

5.7 Real data ingestion & integrations — [MOCK]

All connectors are simulated. **Required (in order):** bulletproof CSV/Excel ingestion with schema validation and clear error reporting -> then **one** real integration (start with SFTP/flat-file or a mid-market API like NetSuite/QuickBooks — **not** SAP). Depth on one real connector beats ten fake ones.

5.8 AI layer — server-side gateway & the moat — [MOCK]

Today the Copilot ships the user's LLM API key to the browser — a security liability and a zero-moat "feature." **Required:** a server-side AI gateway (keys never touch the client), then the actual differentiator — proprietary, data-compounding decision loops (§8), not a chatbot.

5.9 API hygiene & security — [MOCK]

No versioning, inconsistent pagination, no rate limiting, CORS/secrets need review. **Required:** `/api/v1` versioning, consistent pagination/filtering contract, rate limiting, secrets management (env + vault, `.env.example`), input validation via Pydantic on every endpoint, and a documented security posture for buyer reviews (path toward SOC 2).

5.10 Scalability & caching — [PARTIAL]

Single-process, no cache, Redis unused. **Required:** stateless API workers behind a load balancer, Redis caching for expensive reads, and a documented scaling story before you promise enterprise SLAs.

---

6. Build strategy — phased roadmap

The governing rule: **make the foundation real, then make the Primary loop real end-to-end on real data, then harden, then differentiate, then expand.** Do not build breadth before the foundation. Each phase has an exit gate (§7).

Phase 0 — Foundation (the floor) · *do this first, no exceptions*

Multi-tenancy (§5.1) · server-side auth + RBAC (§5.2) · PostgreSQL + Alembic (§5.3) · test scaffold + CI (§5.5) · basic observability (§5.6) · secrets/env hygiene (§5.9). *Outcome:* you can safely onboard two separate customers with isolated data and real logins. **Without this you cannot have a second paying customer — it is the gate to revenue.**

Phase 1 — The Real Primary Loop (your sellable MVP)

Harden Demand -> Diagnostics/FVA -> Inventory -> Consensus **on real uploaded data, with every mock removed from this path.** Add async batch forecasting (§5.4), real CSV/Excel ingestion with validation (§5.7), persistent forecast-vs-actual history so FVA/accuracy is genuine, and a fast onboarding ("time-to-first-forecast in a day"). *Outcome:* a design partner uploads their history and gets real, trusted forecasts + real working-capital/inventory output. **This is the product you sell.**

Phase 2 — Productionization

Security hardening + API hygiene (§5.9) · Sentry + metrics + alerting (§5.6) · performance/caching (§5.10) · one real integration (§5.7) · error handling & data-quality pipeline · begin SOC 2-readiness documentation. *Outcome:* it survives an enterprise security review and real production load.

Phase 3 — Differentiation (earn "AI-native")

Server-side AI gateway (§5.8) · agentic planning workflow (drafts consensus, flags exceptions, proposes replenishment — planner approves) · data-compounding accuracy loop that improves per customer (§8). *Outcome:* a moat that isn't a copyable API wrapper.

Phase 4 — Expansion (breadth, finally)

Turn [PARTIAL] modules real in order of expansion value: **S&OP/IBP -> Executive Analytics -> Pricing -> Retail -> Supplier Portal.** Each only after it's demanded by paying customers. *Outcome:* land-and-expand ACV growth on a real platform.

---

7. Definition of "production-ready" (exit gates)

A module/phase is not "done" until:

- Runs on real tenant data with **no `mock`/`hash`/`random`** in the code path.
- Every endpoint requires auth and enforces tenant isolation + RBAC server-side.
- Covered by automated tests; CI is green; core math has regression tests.
- Errors are handled, logged, and tracked (Sentry); no silent failures.
- Expensive operations run async with visible status; no request timeouts.
- Input is validated; bad data produces clear, actionable errors.
- Metrics have a single source of truth (no duplicate/contradicting numbers across modules).
- A new customer can be onboarded without code changes.
- Documented (API + runbook) accurately — no aspirational claims.

Rule: the README and status docs must always reflect the *code's* reality grade. No feature is described as "Strong" unless it passes this gate. Re-grade §4 and §10 at every phase exit.

---

8. The AI moat (make "AI-native" true)

"AI-native" is currently a claim you cannot defend. Earn it by owning what an LLM API can't give a competitor:

1. **Data-compounding forecasting** — accuracy improves with each customer's history + planner overrides (FVA feedback loop). Competitors calling the same API can't replicate your accumulated signal.
2. **Agentic planning, not chat** — an agent that drafts the consensus forecast, surfaces exceptions, and proposes replenishment/PO actions for planner approval. Decision automation is the value; conversation is the interface.

3. **Server-owned inference** — you host the prompts/orchestration IP; the customer never brings a key. AI is a capability you own, not a setting they configure.

Until this exists, market as **"the modern, planner-native planning platform for the mid-market"** and treat "AI-native" as the roadmap you're actively building — never as a diligence-call claim.

---

9. Risk register

Risk	Severity	Mitigation
Open API / no auth exposed to a real customer	**Critical**	Phase 0 auth before *any* external access
Cross-tenant data leak	**Critical**	Phase 0 multi-tenancy + row-level isolation, test it explicitly
Docs overclaim -> credibility loss in diligence	High	This charter replaces the README; keep grades honest
Solo-founder execution bandwidth	High	Ruthless phase discipline; add technical co-founder/first eng
Forecasting refactor breaks the one real asset	High	Regression tests on math *before* touching it (§5.5)
Breadth creep (building [MOCK] modules early)	High	Freeze Tertiary; enforce §6 gating
Synchronous forecasting times out on real data	Medium	Phase 1 async jobs (§5.4)

---

10. Quick-reference: what to build next

Right now (Phase 0): `organization_id` on every table -> server-side auth on every endpoint -> Postgres + Alembic -> CI + first tests. Nothing else.

Then (Phase 1): real Primary loop (Demand -> FVA -> Inventory -> Consensus) on real data, async, mock-free.

Freeze until later: BI, Execution/Connectors, Supplier Portal, Digital Twin, Pricing, Retail, Workforce, Finance planning — keep for demos, stop building.

One-line mandate: *Make one thing real end-to-end before making anything else broad. Depth in the Primary loop on a real, multi-tenant, tested foundation is the entire job until a customer pays.*

Living document — re-grade §4 and update §6 progress at every phase exit. When you're ready, I can convert any phase into a detailed engineering spec with tickets, or draft the multi-tenancy + auth implementation plan (Phase 0) in full technical detail.